

Adnovum Managed Services (AMS) on Azure

Our roadmap to implementation

Daniel Hogg, Head of Technology Innovation

Vitor Bernardo, Head of Architecture Public Sector

Bern, 12 May 2026



Cloud Native Bern Meetup

adnovum.com

Agenda

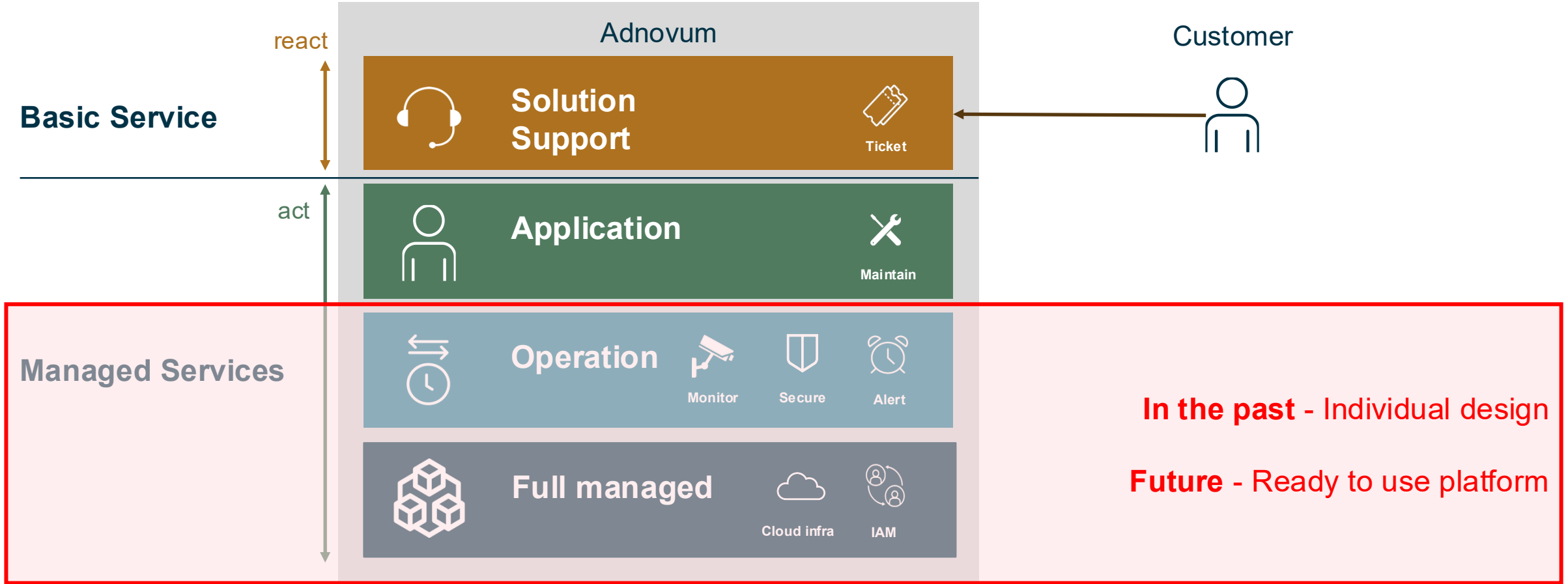
1. Motivation for an Adnovum Managed Services (AMS) platform on Azure
2. Challenges
3. Solution design
4. Context of Sovereignty
5. Takeaways



Motivation for an Adnovum Managed Services (AMS) platform on Azure

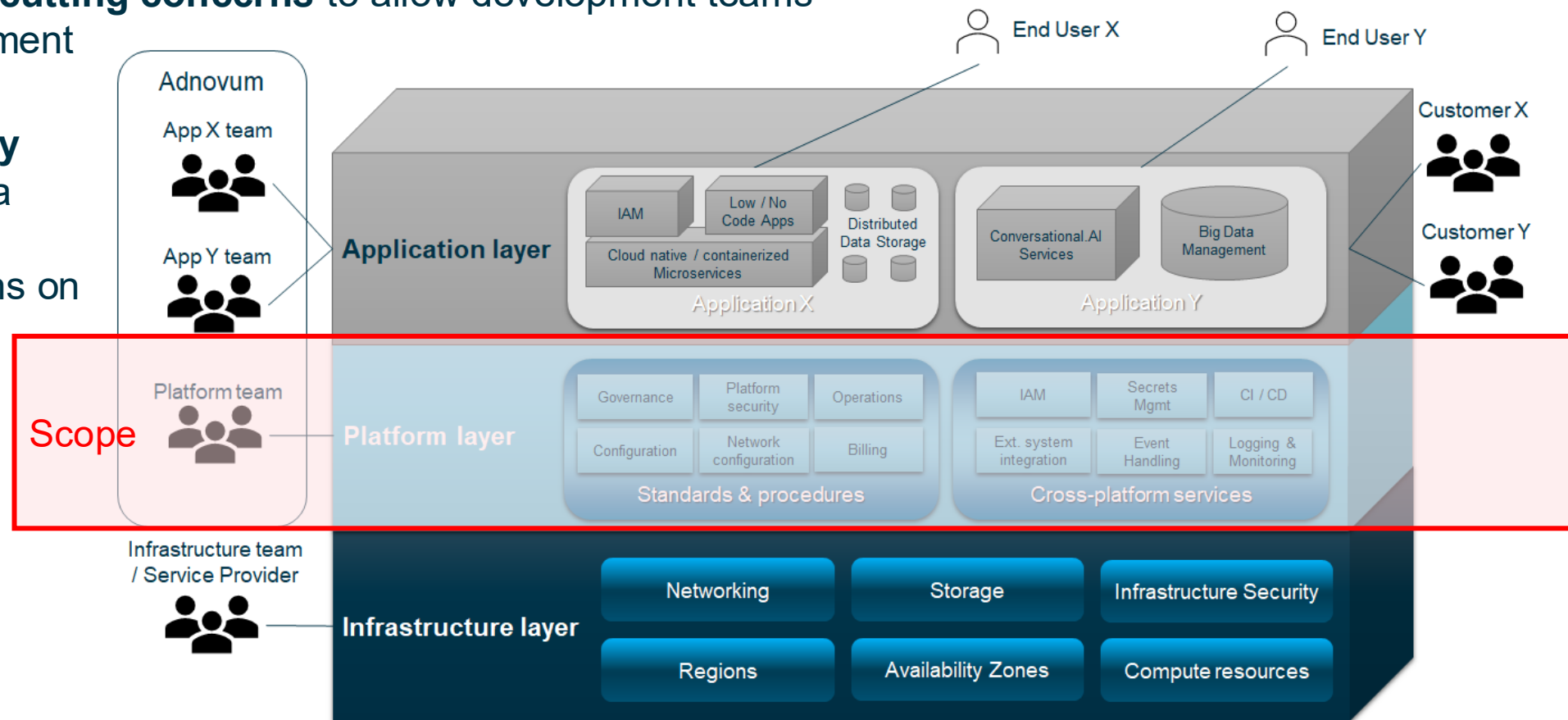


Need for more supplier responsibility, but Adnovum didn't provide a standardized platform



Goals

- Offer a **technical blueprint** (= Minimum Viable Product) for a cloud platform that can be used by Adnovum to host applications for customers in a **secure, reliable** and easy to **maintain** manner
- **Standardize cross-cutting concerns** to allow development teams to focus on development
- **Leverage scalability effects** by allowing a single team manage (multiple) applications on a single platform for multiple customers



Challenges



Main challenges

1. What's the **purpose** of the platform layer?
2. Where to draw the lines between different **roles and responsibilities**?
3. How to **separate applications** logically and physically?
4. How to ensure **security** overall and among different layers?
5. How to handle **identity and access management (IAM)**?
6. How to do **monitoring and observability**?
7. How to keep the solution **maintainable**?



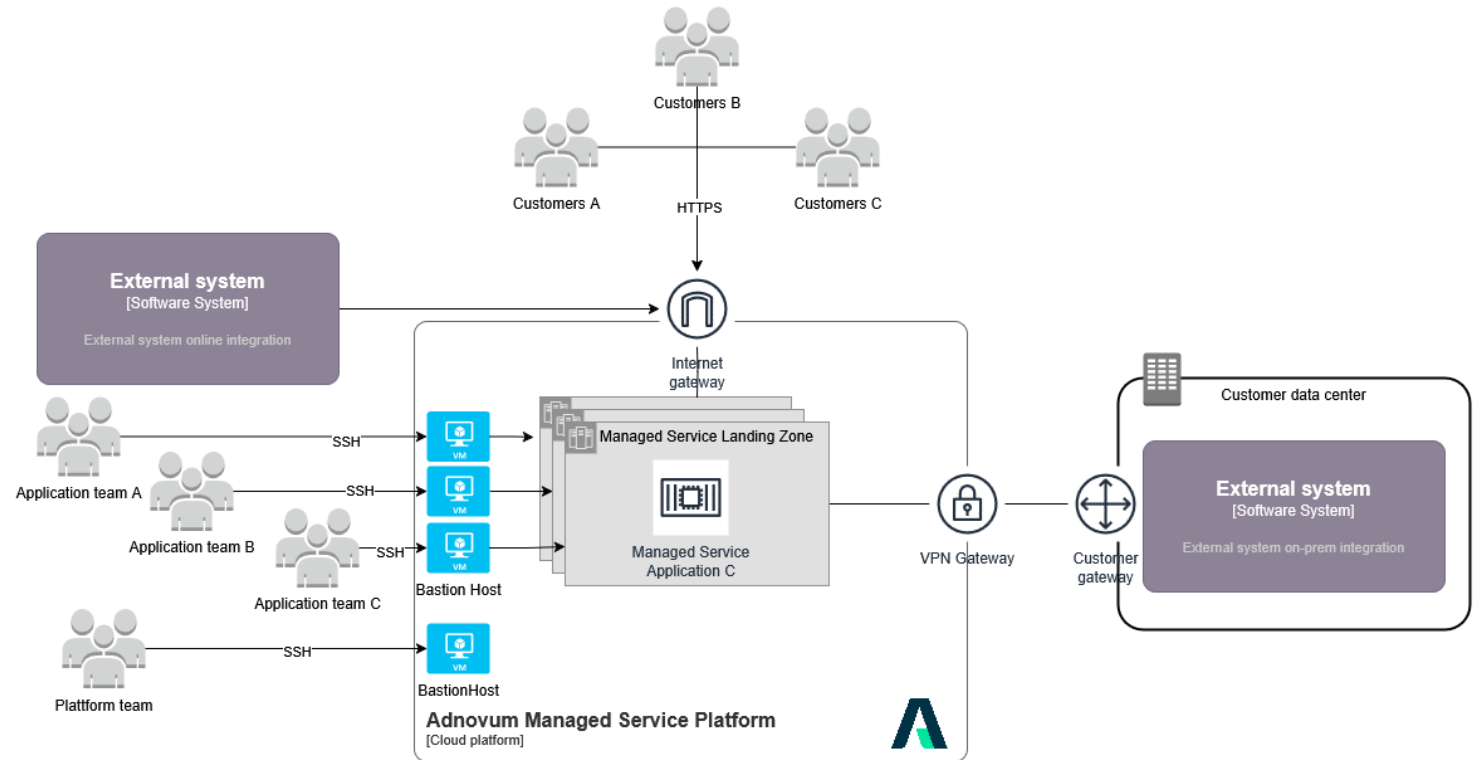
Solution design



The purpose of the platform layer matters

Hosting of workloads in a **standardized way**, ensuring consistency in terms of

- compliance and governance
- automated provisioning
- security and access control
- availability and capacity management
- monitoring and observability
- incident handling and response



We kept the context as specific as possible

User group focus

Target
stakeholders



Application teams



Platform team

Use cases

SaaS cloud solutions
for customers



Blueprint for customer's
cloud environment



Security baseline

Compliance
requirements



Microsoft Security Benchmark
and ISO27001

Supported hosting technologies

Cloud
platform



Workload
orchestration



K8s



Azure App Service

Infrastructure
as code



Terraform

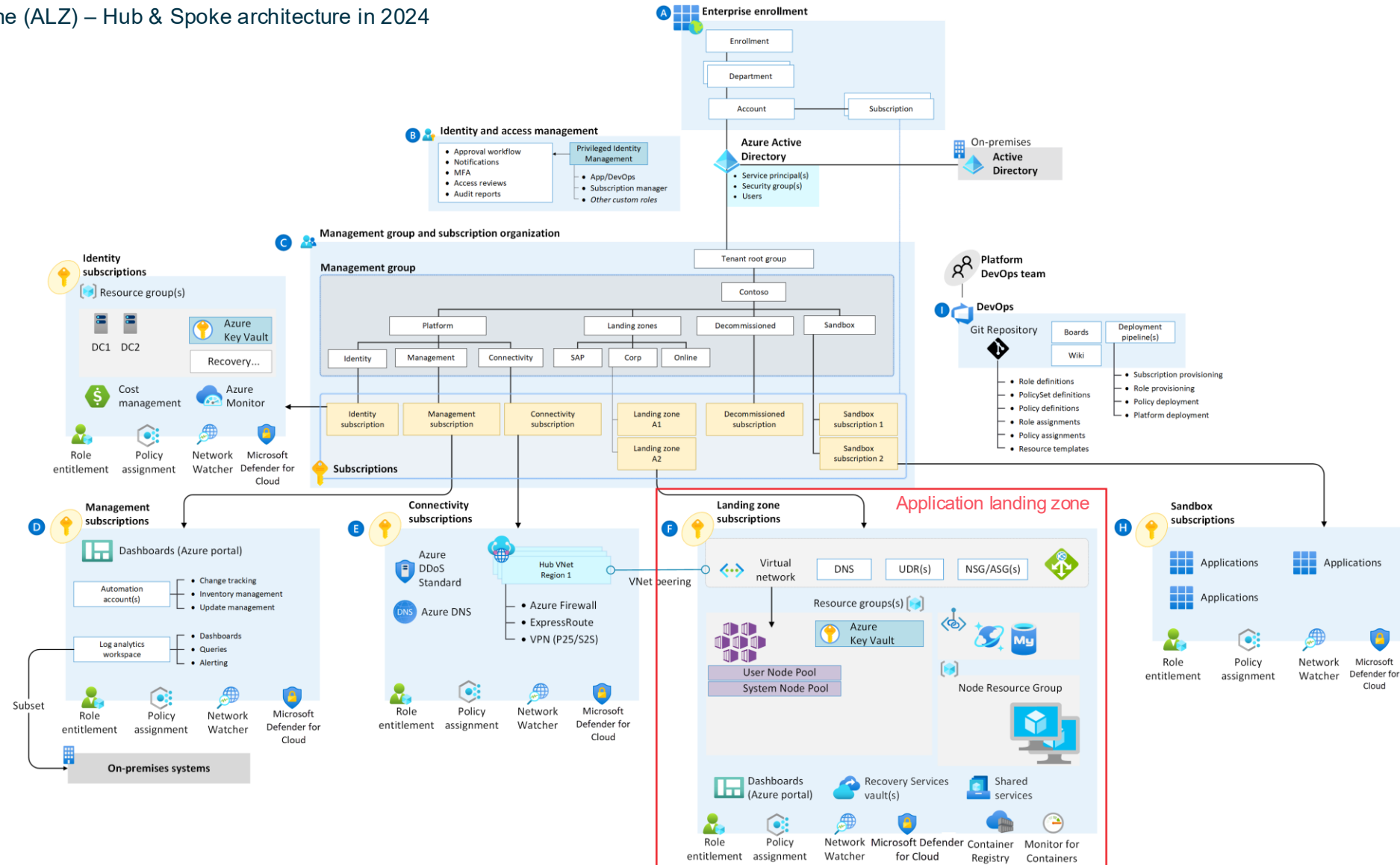
We clarified roles and responsibilities early on

	Application team	 Admin Developer	User management Maintain, monitor and fix workloads, ensure app specific security posture, on-call for app specific failures
	Platform team	 Admin SRE	User management and all other rights of SRE Maintain platform, monitor and ensure service availability, security baseline, emergency response and capacity planning
	Infrastructure provider	 Support	Infrastructure availability and support

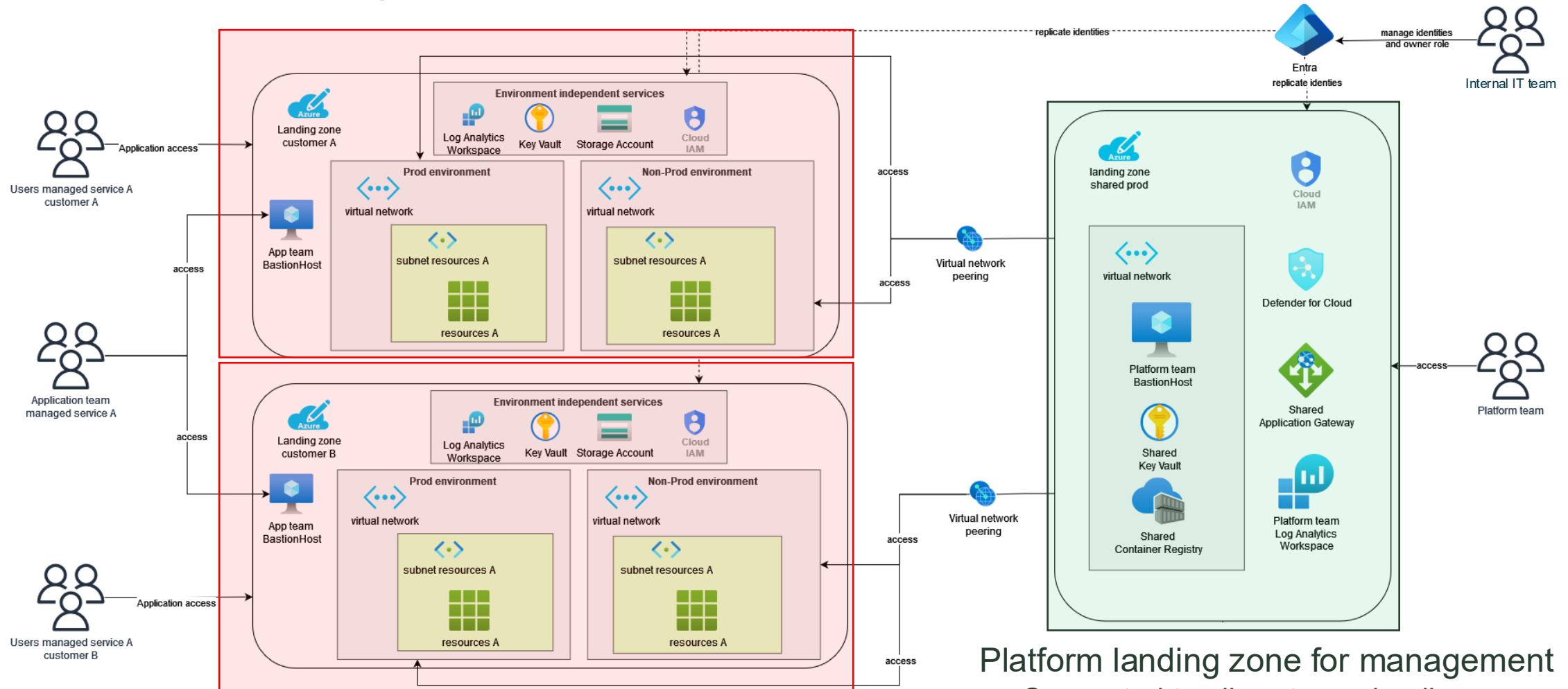


We considered the ideal world for structuring the platform

Azure landing zone (ALZ) – Hub & Spoke architecture in 2024



But chose a segmentation that suits our needs



Secure Landing Zone per customer

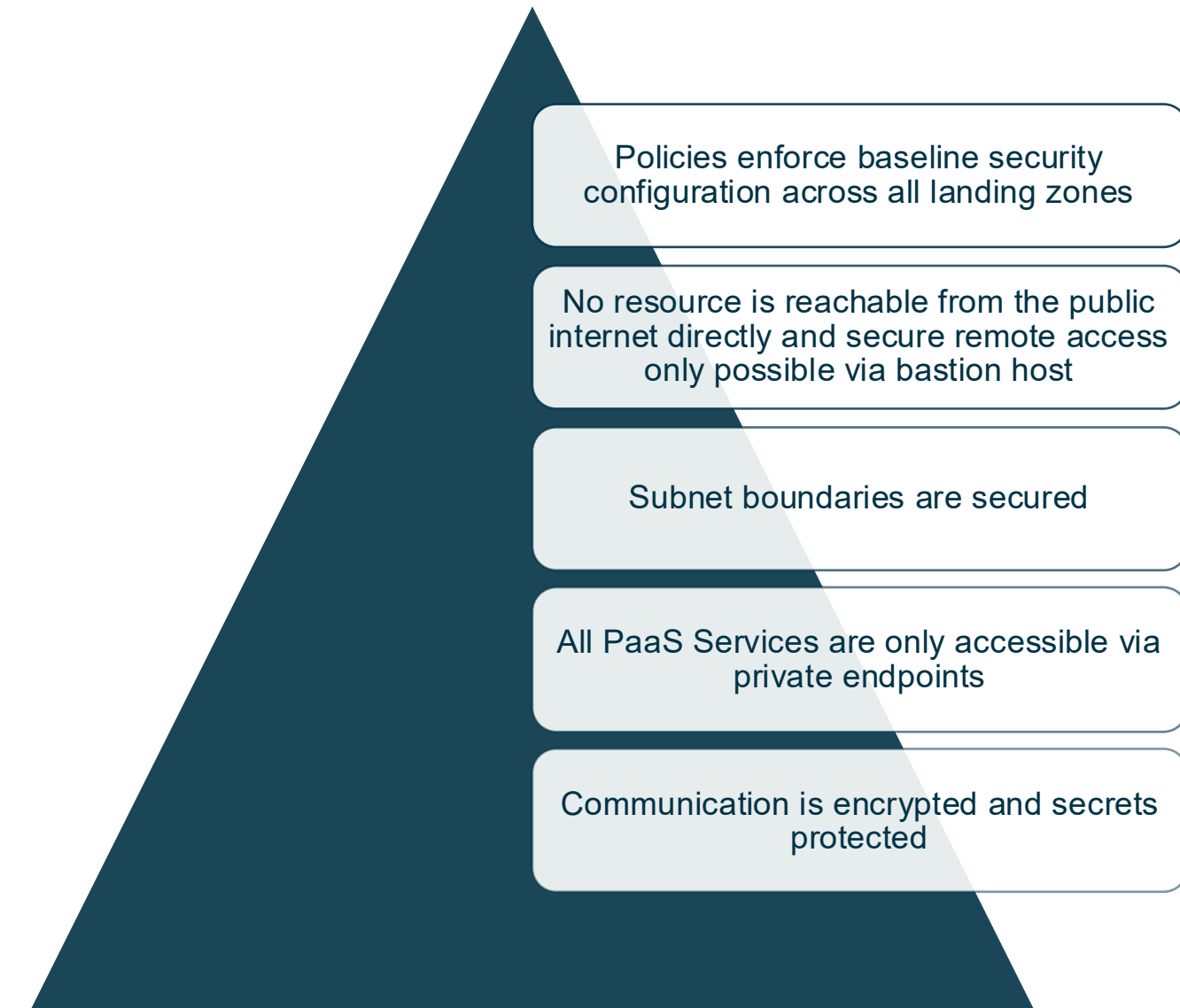
- Divided into separate environments & subnets
- Secure access & communication via bastion host

Platform landing zone for management

- Connected to all customer landing zones
- Central monitoring of resources
- Central security posture
- Central onboarding



AMS applies a *zero trust by design* security philosophy



Policies enforce baseline security configuration across all landing zones

Threat detection is done using **Defender for Cloud** for all services

No resource is reachable from the public internet directly and secure remote access only possible via bastion host

Public internet traffic is monitored and actively blocked via **Application Gateway with WAF**

Subnet boundaries are secured

Network Security Groups (NSGs) are deny-by-default and require explicit allow rules

All PaaS Services are only accessible via private endpoints

Service to service authentication uses **managed identities** everywhere

Communication is encrypted and secrets protected

VNet-only accessible **Key Vault** used as single source of truth for all sensitive values



Every identity is short-lived and scoped to least privilege

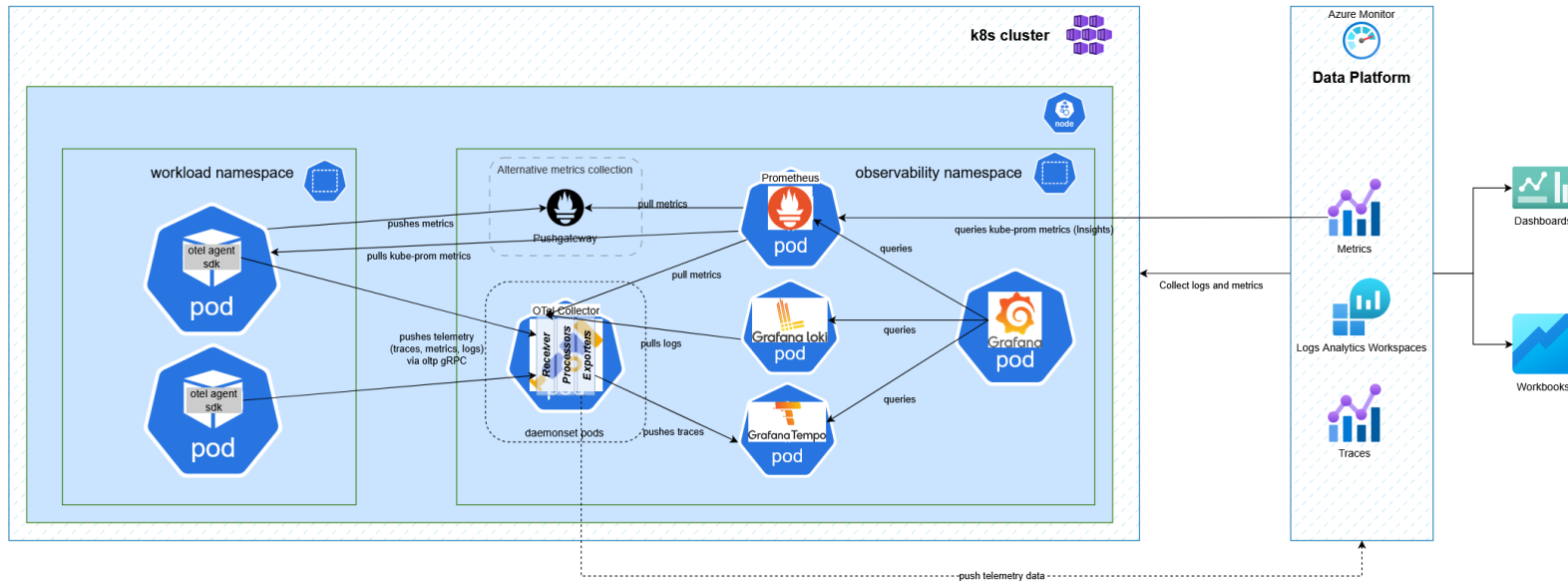
Core IAM Principles for the managed service platform

- The platform layer is designed so that **no service authenticates using passwords, client secrets, or API keys**
- Every identity either uses **token-based federation** (CI/CD) or a **managed identity** (runtime workloads)
- The only passwords at platform layer are database admin credentials **stored exclusively in Key Vault**
- Admin credentials are **never used for day-to-day operations**
- All human admin work is done **via Bastion VMs**
- The **Bastion VMs** themselves require Azure AD authentication via managed identity for resource access



We operate a dual-layer observability model

Prometheus/Grafana stack gives teams a unified, vendor-neutral view across **application** metrics, logs, and traces in a single pane of glass, while **Azure Monitor captures the platform layer** with a full audit trail



Layer 1: Application layer observability

AMS collects telemetry from application workloads running on the platform with application metrics, logs and traces for runtime and incident analysis

Layer 2: Platform layer observability

AMS collects telemetry from infrastructure in log analytics workspaces to raise infrastructure- and security-relevant events



Finally, AMS is kept maintainable through automation, modularity, and enforced standards

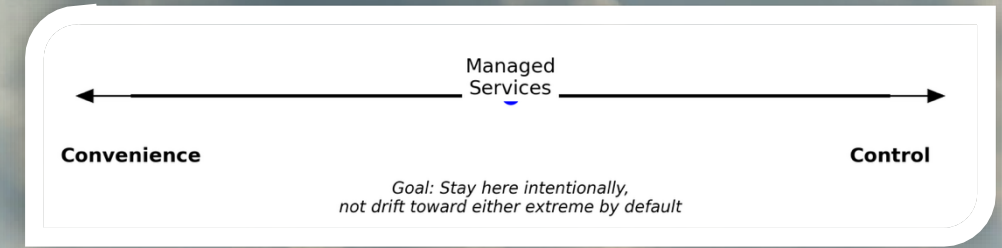
Key measures include

- Consistent **naming and tagging** of resources supports filtering for cost allocation, compliance audits and scoping
- Version control and reproducibility of resources through **IaC** (Terraform) and **automated CI/CD** (GitHub Actions)
- Modular architecture with **consistent landing zones and building blocks** for new customers
- Platform-wide **policies and automatic compliance checks** preventing security drifts
- **Identity-bound resource management** via Azure AD avoiding credential management
- Single **connectivity pattern** through private endpoints
- Standardized **observability**



Digital Sovereignty context

1. Sovereignty is not about avoiding the cloud. It is about avoiding **irreversible decisions / dependencies**
2. **Managed services** can support sovereignty if portability, transparency, and exit feasibility are evaluated upfront
3. **Transparency matters**: runbooks, architecture decisions, and operational knowledge must be **accessible to the client**, not locked inside the provider



<https://www.adnovum.com/blog/digital-sovereignty-sustainable-architecture-is-the-real-challenge>



Takeaways

1. Invest into **landing zone design** as this will determine the level of flexibility in terms of access rights, workload separation and billing process
2. Costs increase with higher level of separation so **bundle commonly used resources** if possible
3. Separate platform deployment into **critical resources** (change rarely) and **dynamic resources** to use a hierarchical stepwise deployment
4. Drawing the lines is important in terms of what is **in-scope vs. out-of-scope**, e.g. 2 variants for monitoring and observability, hybrid support etc.
5. Security requires **constant (re-)work** as security posture management system raises new findings regularly



Get in touch for more information



Daniel Hogg

Head of Technology Innovation
Adnovum AG

daniel.hogg@adnovum.ch



Vitor Bernardo

Head of Architecture Public Sector
Adnovum AG

vitor.bernardo@adnovum.ch

